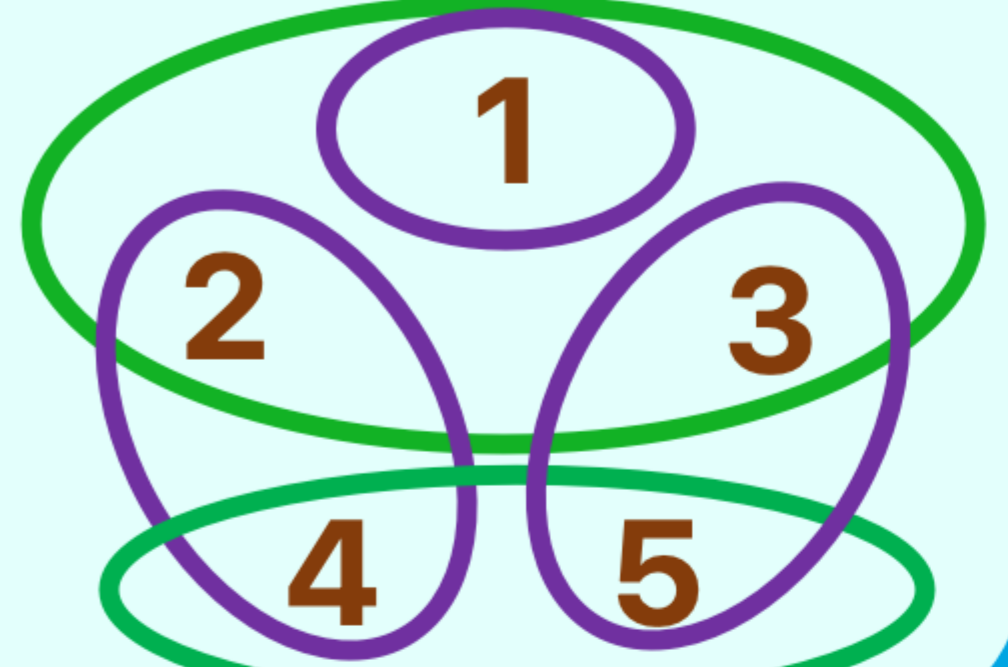


Subset Selection

Subset Nums

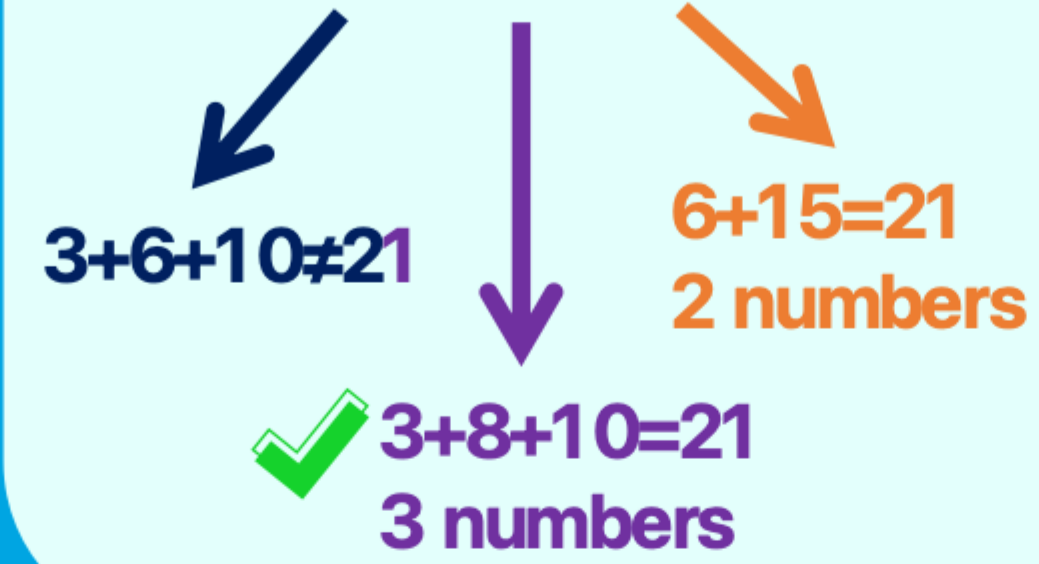
3 → 2



Set Cover Problem

Target Sum 21

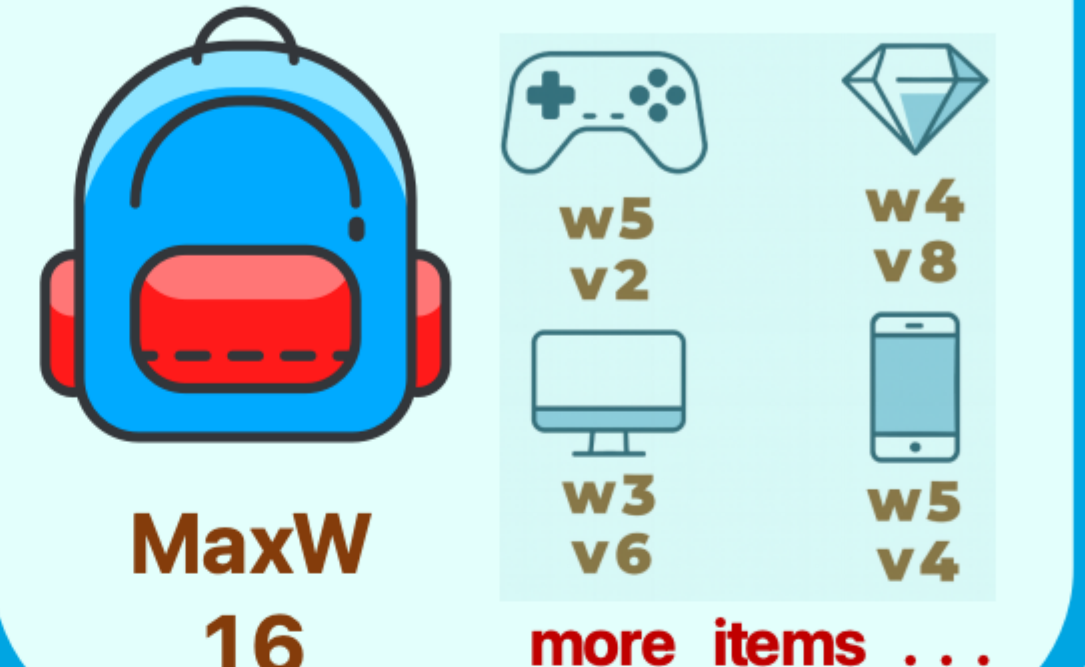
{ 3, 6, 8, 10, 15 }



Subset Sum Problem

Total Value

2+6+4 (w13<16) → 2+8+4 (w14<16)



Knapsack

Path Planning

Path Len

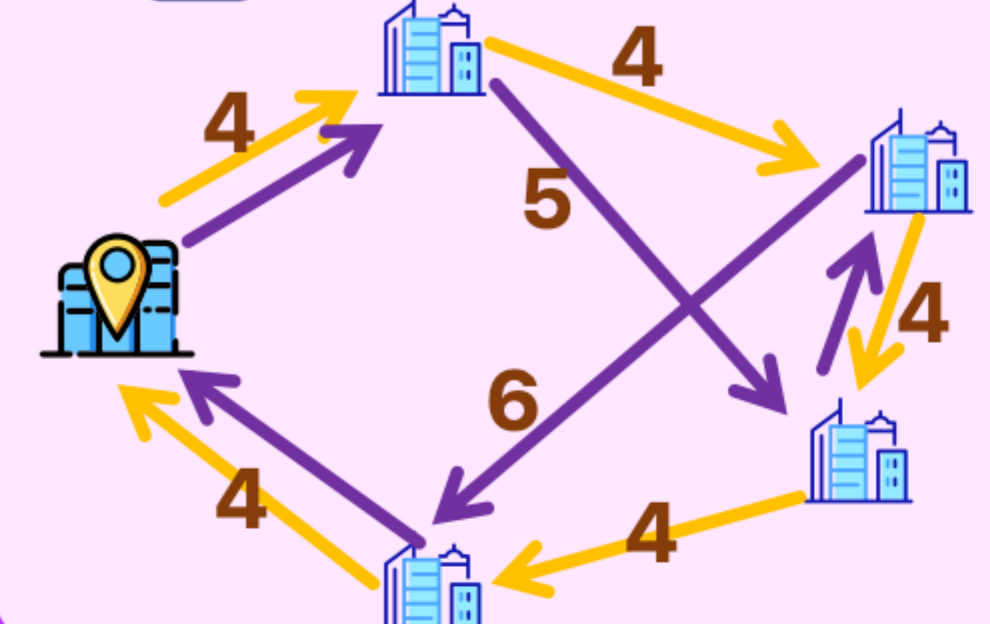
3 → 5



Hamiltonian Cycle Problem

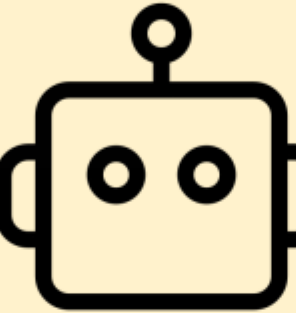
Path Cost

23 → 20



Traveling Salesman Problem

Question: Your objective is to find a subgraph within a given graph that contains a Hamiltonian circuit, which is a path that visits every vertex exactly once and returns to the starting point. The goal is to **maximize** the number of vertices included in the Hamiltonian circuit.



<think>To solve this hamiltonian circuit problem, we need to... 1. Start from a Random Vertex, Let's start from... 2. Find a Valid Small Subgraph, From vertex 1, we can move to vertices 2, 3, 4; we expand to 2... 3. Expand Further...</think>

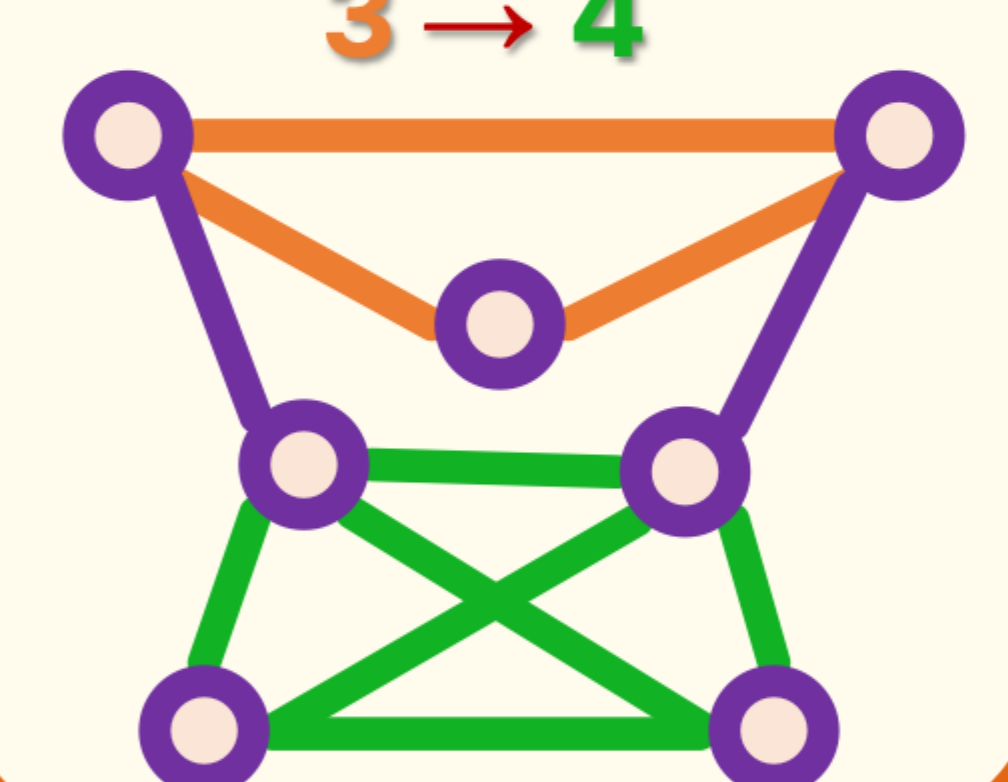
Answer : [1, 2, 3, 1] **1 and 3 are not connected**
Answer : [1, 2, 4, 1] **NOT OPTIMAL**



Graph Clustering

Clique

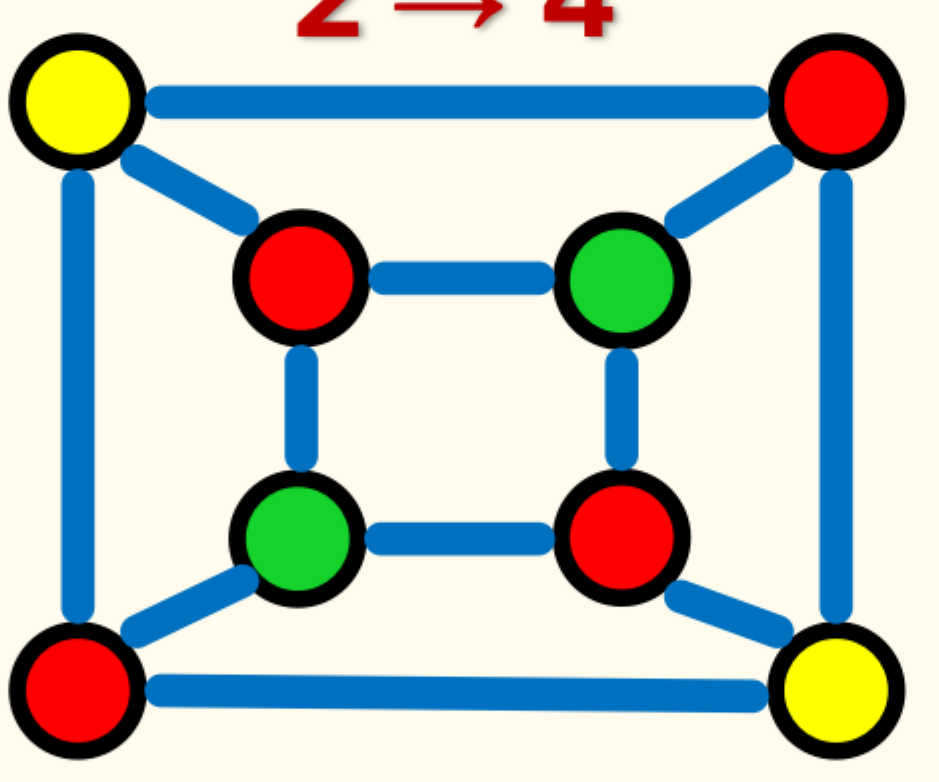
3 → 4



Maximum Clique Problem

Set Size

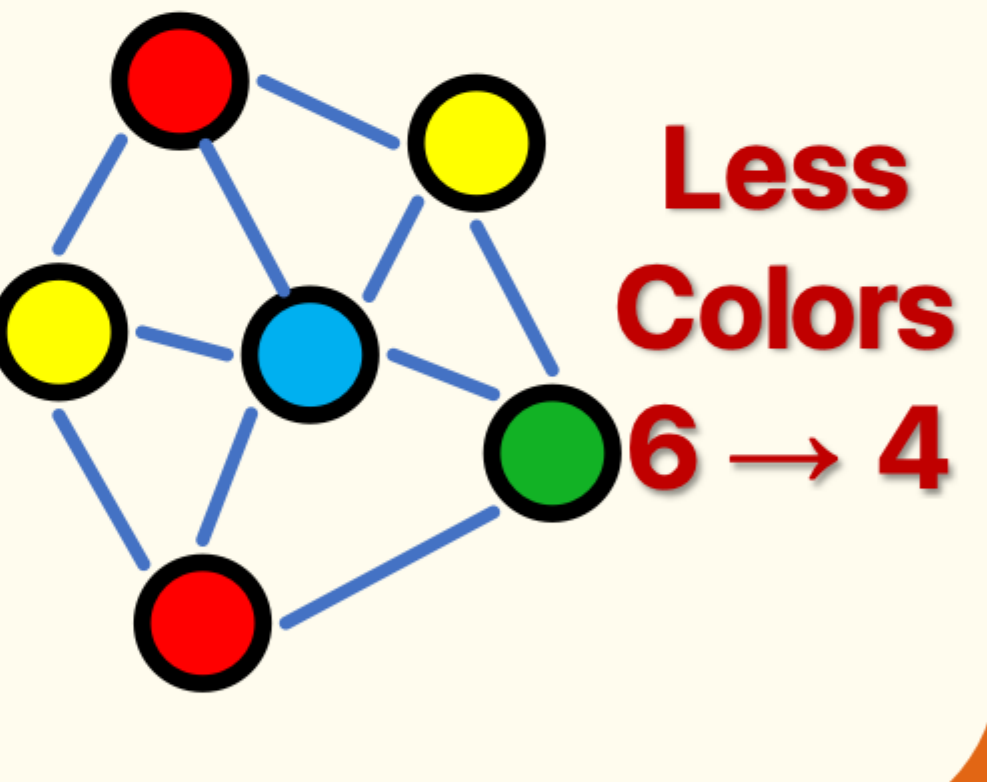
2 → 4



Maximum Independent Set

Less Colors

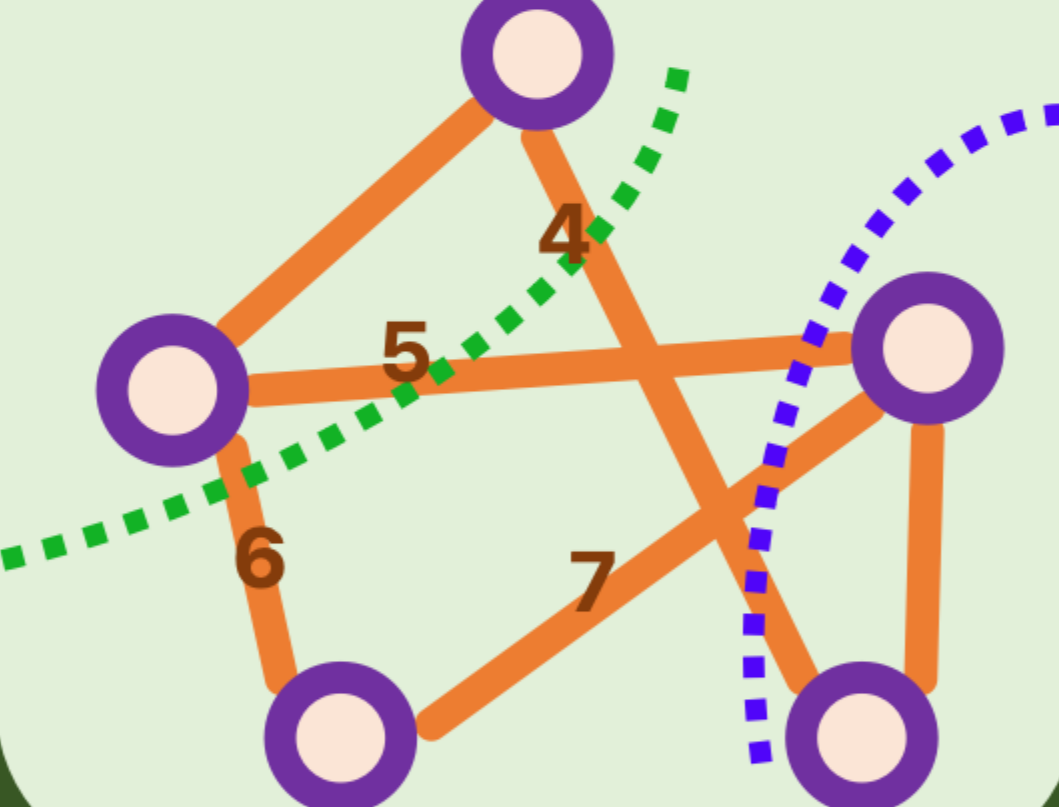
6 → 4



Graph Coloring Problem

Graph Partitioning

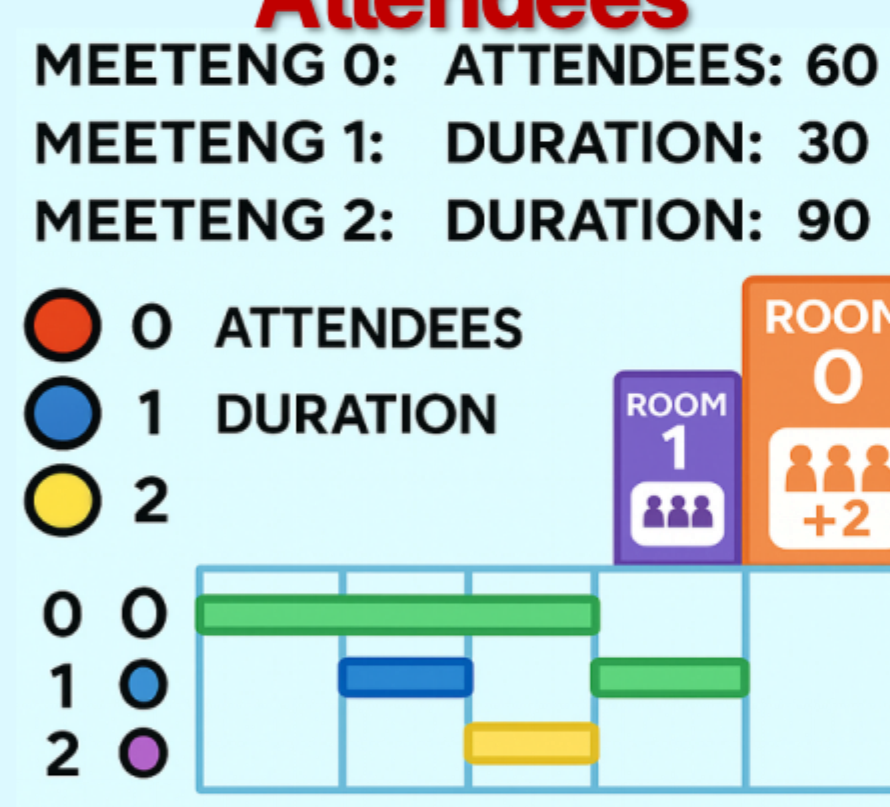
4+5+7 → 4+5+6



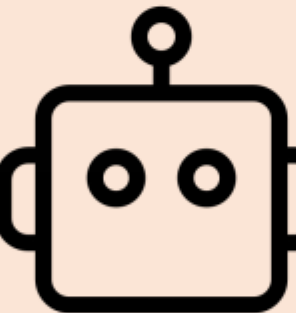
Balanced Minimum Bisection Problem

Resource Scheduling

Maximizing Attendees



Meeting Scheduling Problem



<think>To solve this hamiltonian circuit problem, we need to... 1. Start from a Random Vertex, Let's start from... 2. Find a Valid Small Subgraph, From vertex 1, we can move to vertices 2, 3, 4; we expand to 4... 3. Expand Further, we can move to 6... 4. **Continue the Path...** 5. Further Expansion... 6. Final Answer: ... 7. **Verify the final solution ...**</think>

Answer : [1, 4, 6, 0, 7, 5, 3, 1] **OPTIMAL ANSWER**



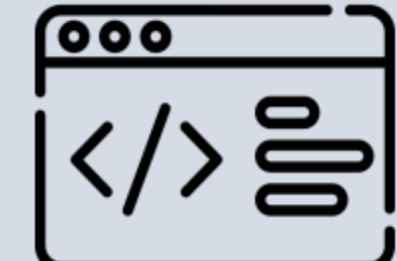
Data Generator

```
def Generator(N_range, density):  
    N = Sample(N_range, density)  
    G = Task_Generator(N)  
    G = Task_Tempate (G)  
    return G_final
```



Solution Validator

```
def Validator(response, G):  
    path = Output_Parser(response)  
    is_valid = Checker(path, G)  
    score = Reward(is_valid, path)  
    return score
```



Heuristic Solver

```
def Heuristic_Solver(G):  
    state = Initialize_State(G)  
    best_state = Search(state)  
    value = Fn(best_state)  
    return value
```

Generalization

